

Why LLMs can't make your code simpler



Based on my [Medium Article](#)

About me



- Working @ Answer.AI
- AI Research & Development Lab
- Founded by:
 - Jeremy Howard (fast.ai, kaggle, ULMFit)
 - Eric Ries (Lean Startup, LTSE, Incorruptible)
- 12 people team
- What is AI good for?
- Extending humans rather than replacing them
- Exploring what's the latest possible with LLMs

The problem: LLMs as sources of complexity

- LLMs make codebases grow complex very fast.
 - Duplicate existing methods
 - Defend against impossible edge cases
 - Optimize too early.
- Keeping codebase concise takes a lot of effort
- Getting worse as LLMs can work alone longer

Couldn't frontier labs fix this?

Reward models for **reducing code complexity** metric

1. Fewer lines of code
2. Less duplication
3. Reduce # of code paths
4. ...

Would that produce simpler programs?

Which code is simpler?

OPTION A

```
class OpenAIClient:
    def complete(self, prompt):
        msgs = prepare(prompt)
        y = call_openai(msgs)
        return with_retries(y)

class AnthropicClient:
    def complete(self, prompt):
        msgs = prepare(prompt)
        y = call_anthropic(msgs)
        return with_retries(y)
```

OPTION B

```
class LLMClient:
    def __init__(self, call):
        self.call_provider = call
    def complete(self, prompt):
        msgs = prepare(prompt)
        y = self.call_provider(msgs)
        return with_retries(y)
```

Then your boss tells you...

We will probably stop supporting Anthropic because it's too expensive

Which one is simpler now?

```
class OpenAIClient:
    def complete(self, prompt):
        msgs = prepare(prompt)
        y = call_openai(msgs)
        return with_retries(y)

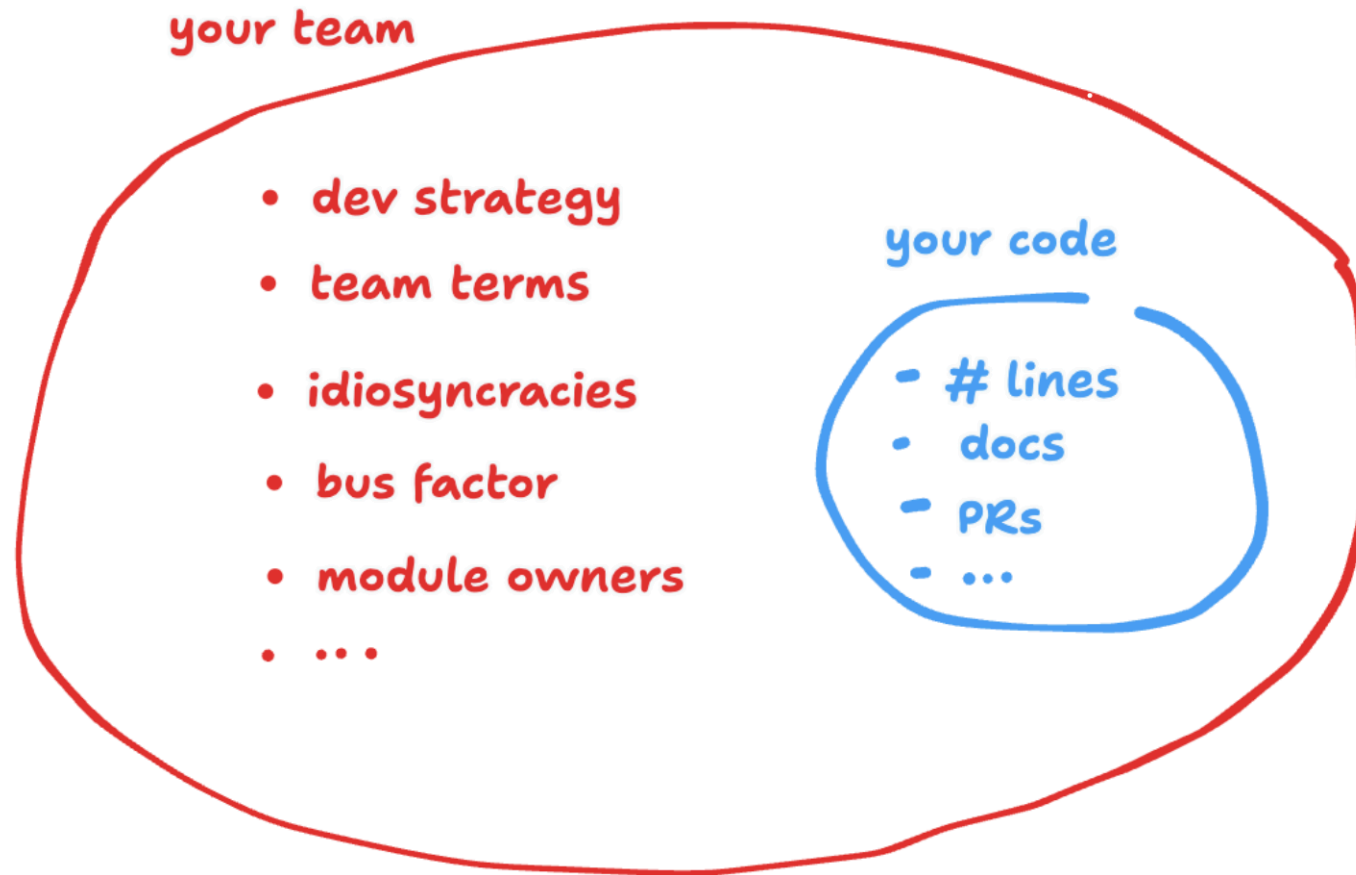
class AnthropicClient:
    def complete(self, prompt):
        msgs = prepare(prompt)
        y = call_anthropic(msgs)
        return with_retries(y)
```

```
class LLMClient:
    def __init__(self, call):
        self.call_provider = call
    def complete(self, prompt):
        msgs = prepare(prompt)
        y = self.call_provider(msgs)
        return with_retries(y)
```

Why is that?

- Our decision changed based on information **beyond** the code has changed.

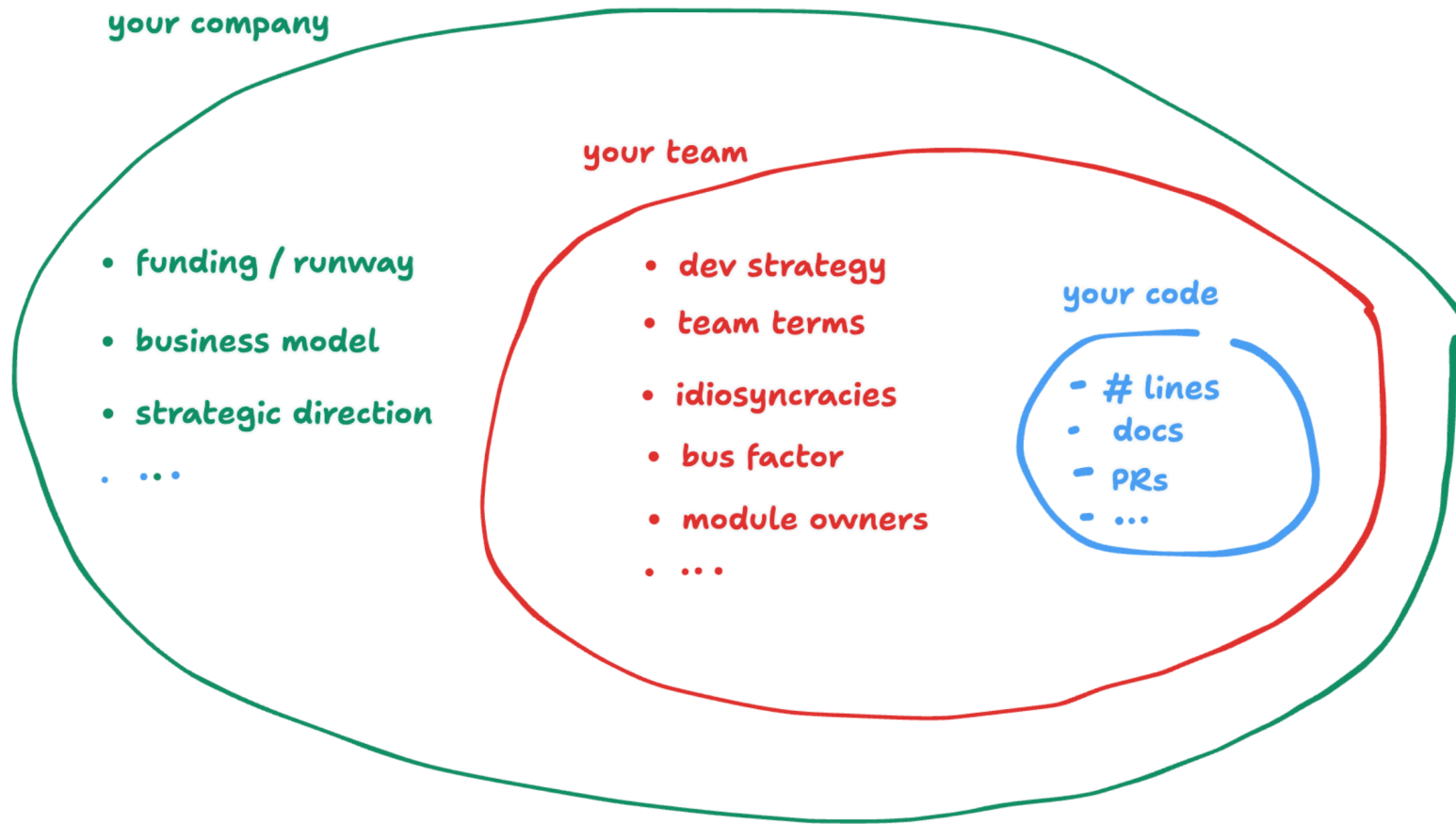
Your code is just a piece of the cake



Then Fable is released...

and the CEO tells you to ship it asap so he can pitch it to investors

Your code is just a piece of the cake

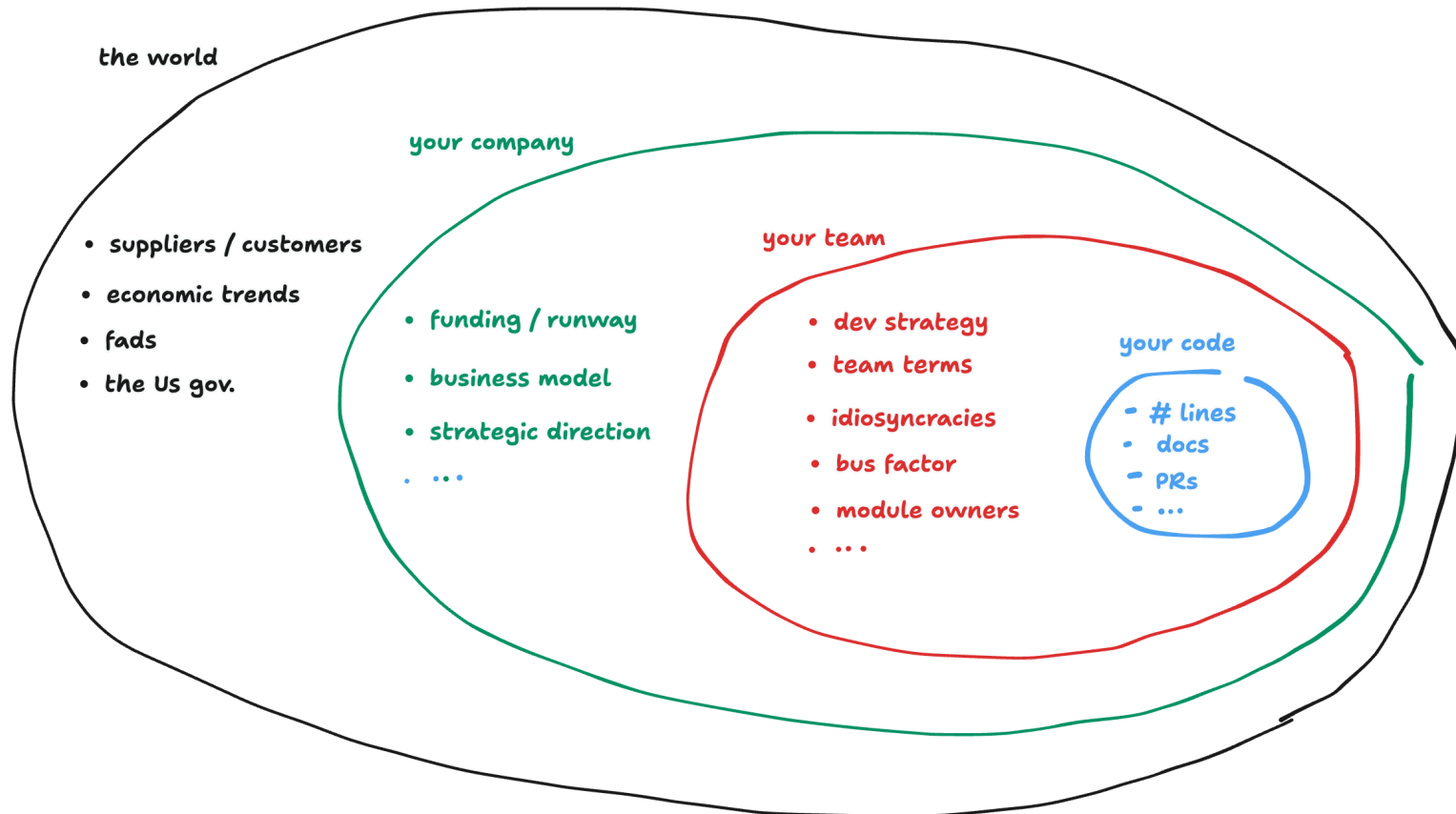


But then Fable is banned by the US...

and you no longer have API access to it.

Your code is just a piece of the cake

A lot of external factors affect the code



What should we call this context?

The Theory

Peter Naur formalized this concept in his *Programming as Theory-building* paper (1985)

- He called this "the Theory".
- This "context" in people's minds **is the program**.
- Code & docs are only downstream artifacts.

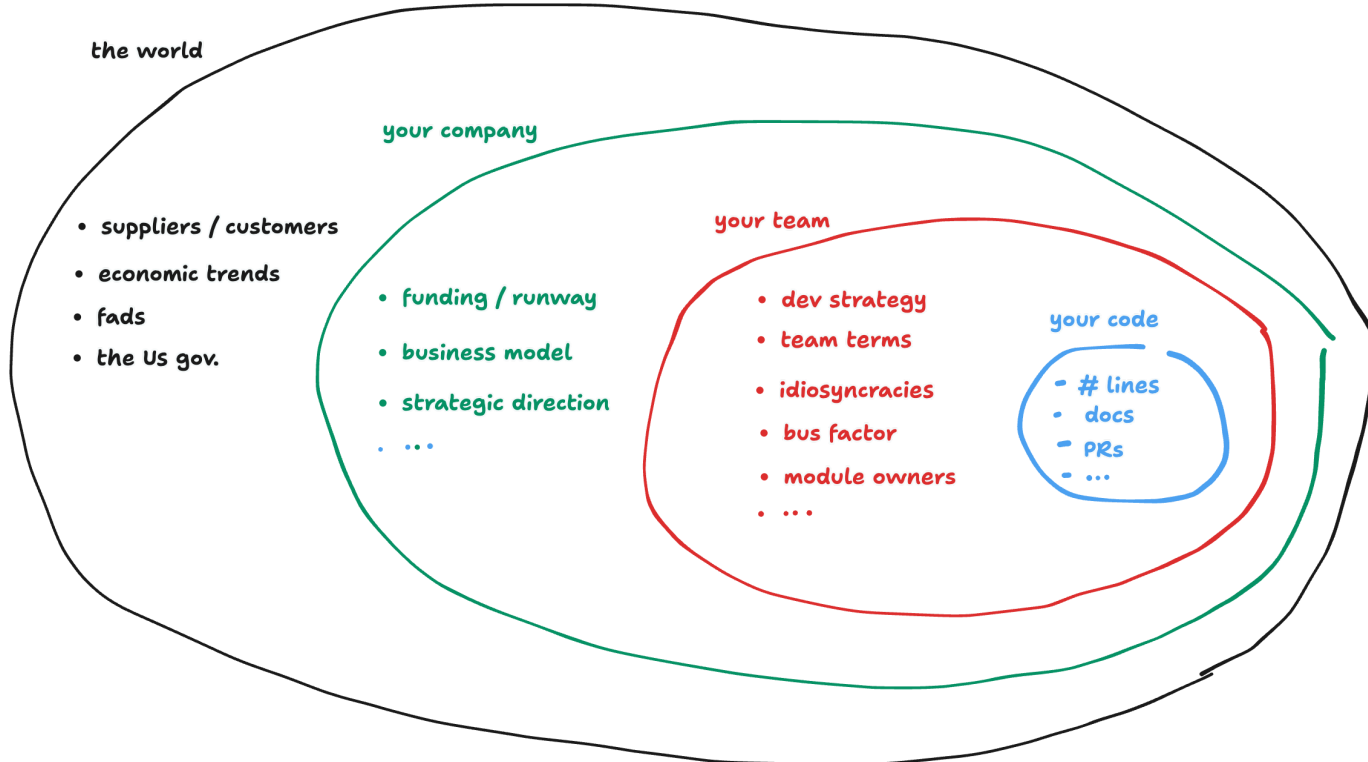


So what?



The Theory

- Code & docs can never fully capture the Theory.
- As soon as they are written they are outdated & incomplete



Recap

- There's no objective metric for code simplicity
- What "best code" means is determined by external factors
- Naur calls this knowledge the "Theory"
- Code & docs can't fully capture the "Theory"
- LLMs lack Theory unless you provide it

How to apply this knowledge? Let's see an example...

The Theory in Practice: Simplifying a billing system

Initial system:

- Subscriptions + manual top ups.
- A lot of book keeping + hard to understand

How we simplified it?

- Frequent discussions
- Users feedback
- What if we only have credit?
- Does Stripe support that?

We spent around 2 months of work exploring

The Theory in Practice: Simplifying a billing system

The final system Theory is a single sentence

Users add their CC on login, and they can top up credits manually or automatically

- Minimal Stripe interaction
- DB-first approach
- 300 LoC of billing logic

The Theory in Practice: Problems

Launch Day Issues

On launch day we noticed India-issued cards can't be charged **off-session**.

So:

- Manual top up: works, users get a 3DS-like approval flow
- Auto top up: **broken**

The Theory in Practice: LLMs

Trying to support indian users:

- Me: how we could support Indian users?
- LLM: Use Stripe subscriptions managed service
- Me: but we just **migrated out** of that!
- LLM: I can revert those changes for you or use managed subscriptions only for indian users.

Solutions:

- A. Revert to initial system
- B. Support both systems
- C. Do nothing

What's your opinion?

What did we do? Nothing.

We did nothing.

- Requiring Indian users to manually top up was ok
- We value more a concise payment Theory

Once I explained that, the LLM agreed: "You're absolutely right!"

But that **is not** the right choice in other situations:

- Revenue matters most -> friction loses India sales, migrate back
- Sales uses Stripe UI -> our DB-first approach is wrong
- ...

It is the right choice for our Theory, ie our team, business plan & env.

Discuss the Theory with your LLMs

Pr: Let's discuss the Theory (capital T, Naur) of this before we touch the code.

Discuss about the Theory with your LLMs

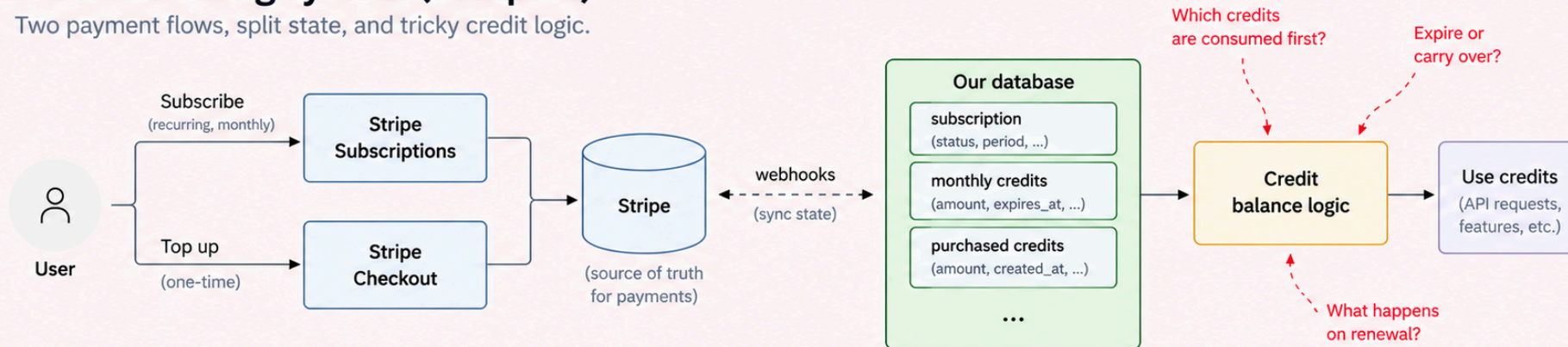
Pr: Let's discuss the Theory (capital T, Naur) of this before we touch the code.

- LLMs know about Naur's Theory
- Useful thinking framework
- Implementation details vs Theory break
- Useful during:
 - Designing features
 - Exploring codebases
 - Reviewing PRs

LLMs can help simplify, but you need to be in charge

Previous billing system (complex)

Two payment flows, split state, and tricky credit logic.



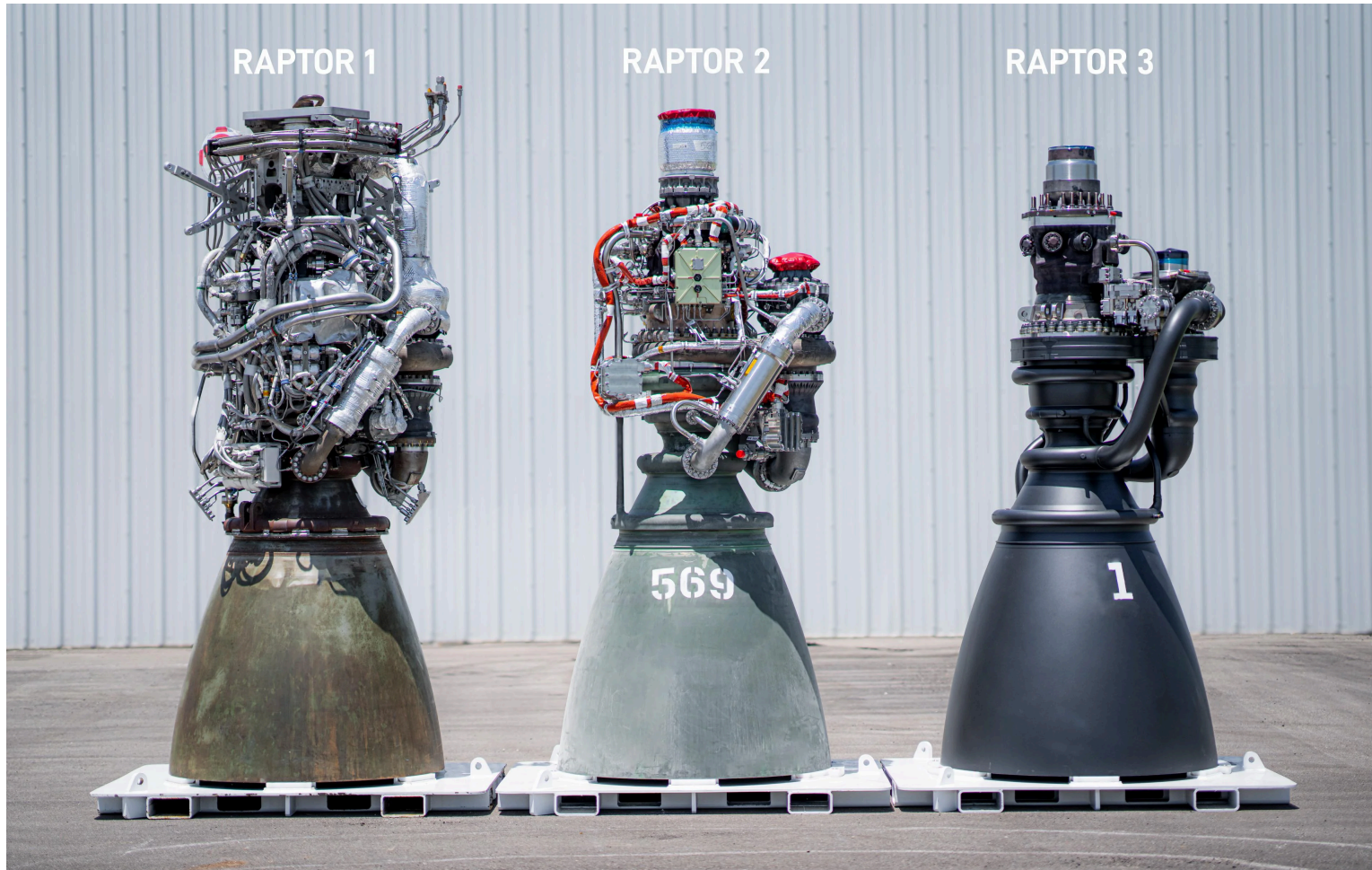
New billing system (simple)

One saved card, one flow, one balance.



Card on file → charge it manually or automatically.

LLMs can help simplify, but you need to be in charge



Talk to you LLMs about the Theory

A program is the Theory held by the people who build and maintain it... at least for now.

Resources:

- github.com/kafkasl/code-complexity-talk
- x.com/pol_avec